Universidade da Beira Interior

Unidade Curricular de Sistemas Distribuídos

Relatório Técnico
Jogo da Forca Multijogador

Autores:
Dinis Gonçalves (52229)
Francisco Pandeirada (52919)

Covilhã, 19 de abril de 2026

Resumo

O presente relatório descreve o desenvolvimento de uma aplicação distribuída baseada no jogo da forca em modo multijogador síncrono. A solução foi implementada em Java, segundo uma arquitetura cliente-servidor, recorrendo a sockets TCP/IP para comunicação entre processos.

O sistema permite a ligação simultânea de vários jogadores a um servidor central, que fica responsável pela lógica do jogo, pela gestão das rondas, pela manutenção do estado global da partida e pela comunicação com os clientes. Cada participante dispõe de um cliente próprio, através do qual acompanha o estado da palavra, o número de tentativas restantes e as informações de jogo enviadas pelo servidor.

Ao longo deste documento são apresentadas a arquitetura da aplicação, o protocolo de comunicação adotado, os mecanismos de sincronização utilizados, as principais decisões de implementação e os testes realizados para validação do sistema.

Conteúdo

1 Introdução **3**

2 Objetivos do Projeto **3**

3 Arquitetura do Sistema **3**

3.1 Visão Geral 3

3.2 Componentes Principais 4

4 Funcionamento Geral do Jogo **4**

4.1 Inicialização do Servidor 4

4.2 Ligação dos Clientes 4

4.3 Escolha da Palavra 5

4.4 Execução por Rondas 5

4.5 Jogadas dos Jogadores 5

4.6 Fim do Jogo 5

5 Estado do Jogo **6**

6 Protocolo de Comunicação **6**

7 Concorrência e Sincronização **7**

7.1 Thread por Cliente 7

7.2 Thread de Gestão da Sessão 7

7.3 Sincronização do Estado Partilhado 7

8 Cliente e Interação com o Utilizador **7**

9 Robustez e Tratamento de Situações Especiais **8**

9.1 Desconexão de Clientes 8

9.2 Rejeição de Novas Ligações 8

9.3 Ficheiro de Palavras Inexistente 8

10 Testes Realizados **8**

11 Compilação e Execução **9**

12 Ligações do Projeto **9**

13 Conclusão **9**

14 Melhorias Futuras **10**

1. Introdução

Os sistemas distribuídos desempenham um papel fundamental no desenvolvimento de aplicações modernas, permitindo a cooperação entre vários processos executados em máquinas distintas ou na mesma máquina, comunicando através de uma rede. Este tipo de aplicações levanta desafios importantes, como concorrência, sincronização, consistência de estado e tratamento de falhas.

No contexto da unidade curricular de Sistemas Distribuídos, foi proposto o desenvolvimento de um jogo da forca em modo multijogador. O objetivo consistiu em implementar uma aplicação cliente-servidor funcional, capaz de permitir a participação de vários jogadores numa sessão síncrona, coordenada por um servidor central.

Este projeto procurou aplicar, num caso prático, conhecimentos relacionados com sockets TCP, programação concorrente em Java, definição de protocolos de comunicação e gestão de estado partilhado.

2. Objetivos do Projeto

O trabalho teve como principais objetivos:

- desenvolver uma aplicação distribuída segundo o modelo cliente-servidor;
- implementar comunicação em rede através de sockets TCP/IP;
- suportar entre 2 e 4 jogadores na mesma sessão;
- coordenar o jogo por rondas síncronas;
- garantir consistência no estado global do jogo;
- permitir que cada cliente envie jogadas ao servidor segundo um protocolo textual;
- tratar o fim do jogo por vitória ou derrota;
- encerrar corretamente as ligações no final da sessão.

3. Arquitetura do Sistema

A aplicação foi estruturada segundo uma arquitetura cliente-servidor. O servidor centraliza toda a lógica da partida, enquanto os clientes representam os jogadores e funcionam como interface de interação com o utilizador.

3.1 Visão Geral

$$\begin{array}{c} \text{Cliente 1} \\ \text{Cliente 2} \\ \text{Cliente 3} \\ \text{Cliente 4} \end{array} \quad \Longrightarrow \quad \begin{array}{c} \text{Servidor Principal} \\ \downarrow \\ \text{GerenciadorDeJogo} \\ \downarrow \\ \text{EstadoJogo} \end{array}$$

Nesta abordagem, o servidor recebe as ligações, gere os jogadores ativos, escolhe a palavra, controla as rondas e decide o resultado final. Os clientes apenas apresentam informação ao utilizador e enviam as jogadas introduzidas.

3.2 Componentes Principais

A solução encontra-se organizada em várias classes, cada uma com uma função específica:

1. **ServidorJogo.java** Classe responsável por iniciar o servidor TCP, aceitar ligações e arrancar a sessão de jogo quando se verificam as condições necessárias.
2. **GerenciadorCliente.java** Classe que gere a comunicação individual com cada cliente. Cada instância corre numa thread própria.
3. **GerenciadorDeJogo.java** Classe responsável pela coordenação da sessão, gestão das rondas, processamento das jogadas e determinação do fim do jogo.
4. **EstadoJogo.java** Classe que armazena o estado global da partida, incluindo a palavra secreta, letras usadas, tentativas restantes e vencedores.
5. **BancoDePalavras.java** Classe que fornece a palavra a adivinhar, tentando ler de um ficheiro externo e usando uma lista interna como alternativa.
6. **ProtocolMessages.java** Classe utilitária que define as mensagens do protocolo textual trocado entre servidor e cliente.
7. **ClienteJogo.java** Classe principal do lado do cliente, responsável pela ligação ao servidor, receção de mensagens e envio de jogadas.

4. Funcionamento Geral do Jogo

O funcionamento do sistema pode ser descrito em várias etapas.

4.1 Inicialização do Servidor

Quando o servidor é iniciado, é criado um `ServerSocket` na porta 12345. O servidor aceita um mínimo de 2 jogadores e um máximo de 4 jogadores por sessão. Existe ainda um tempo de espera antes do início da partida, durante o qual os jogadores podem entrar na sessão.

Se não for atingido o número mínimo de jogadores até ao fim do tempo de espera, o servidor encerra. Caso contrário, a partida é iniciada com os jogadores disponíveis.

4.2 Ligação dos Clientes

Cada jogador executa o cliente e estabelece ligação ao servidor. Após a ligação, o cliente recebe uma mensagem de boas-vindas com o identificador atribuído pelo servidor.

Enquanto o jogo não começa, os jogadores permanecem em espera. Quando a sessão é iniciada, o servidor envia a todos os clientes a informação necessária para arrancar a partida.

4.3 Escolha da Palavra

A palavra secreta é escolhida de forma aleatória a partir do banco de palavras. A aplicação tenta obter as palavras de um ficheiro externo, o que facilita futuras extensões. Se tal não for possível, é utilizada uma lista interna definida no código.

4.4 Execução por Rondas

O jogo decorre por rondas síncronas. Em cada ronda, o servidor envia a todos os clientes:

- a máscara atual da palavra;
- o número de tentativas restantes;
- as letras já utilizadas;
- o número da ronda.

Depois disso, cada jogador dispõe de um intervalo de tempo para submeter a sua jogada. No final da ronda, o servidor processa as jogadas recebidas e atualiza o estado global da partida.

4.5 Jogadas dos Jogadores

Em cada ronda, cada jogador pode enviar:

- uma única letra; ou
- a palavra completa.

As comparações são feitas sem distinção entre maiúsculas e minúsculas, uma vez que os dados são normalizados internamente. Se a jogada for correta, o estado do jogo é atualizado. Se a jogada estiver errada, o número de tentativas restantes é decrementado.

4.6 Fim do Jogo

A partida termina quando ocorre uma das seguintes situações:

- a palavra é descoberta, existindo um ou mais vencedores;
- o número de tentativas restantes chega a zero.

No final, o servidor envia uma mensagem de término a todos os clientes e encerra as respetivas ligações.

5. Estado do Jogo

O estado global da partida é mantido na classe `EstadoJogo`. Esta classe centraliza toda a informação necessária para garantir coerência no funcionamento da aplicação.

As principais informações armazenadas são:

- palavra secreta;
- letras já utilizadas;
- tentativas restantes;
- lista de vencedores;
- indicação de fim de jogo.

Esta classe disponibiliza métodos para processar jogadas, revelar a máscara atual da palavra, obter as letras usadas e verificar se a partida já terminou.

A separação do estado do jogo numa classe dedicada simplifica a organização do código e facilita a sincronização entre threads.

6. Protocolo de Comunicação

A comunicação entre cliente e servidor foi implementada através de um protocolo textual simples, em que cada mensagem corresponde a uma linha de texto. As mensagens são separadas por ponto e vírgula, o que facilita a análise e a interpretação do conteúdo.

Mensagem	**Descrição**
`BEM_VINDO;id;total`	Identificação do jogador e número total de jogadores.
`INICIO;mascara;tentativas;tempo`	Início da sessão de jogo (tempo em ms).
`RODADA;numero;mascara;tentativas;usadas`	Informação de uma nova ronda.
`ESTADO;mascara;tentativas;usadas`	Estado atualizado da partida.
`FIM_VITORIA;ids;palavra`	Fim do jogo com vencedores.
`FIM_PERDA;palavra`	Fim do jogo sem vencedores.
`CHEIO`	Servidor cheio ou jogo já iniciado.
`LOBBY_TIMEOUT`	Tempo de espera no lobby expirado.
`jogada;texto`	Jogada enviada do cliente para o servidor.

Tabela 1: Mensagens utilizadas no protocolo de comunicação

A opção por um protocolo textual permitiu simplificar os testes, a depuração e a visualização das mensagens trocadas entre os vários componentes.

7. Concorrência e Sincronização

Uma vez que o servidor precisa de tratar vários clientes em simultâneo, a aplicação recorre a programação concorrente.

7.1 Thread por Cliente

Cada cliente ligado é tratado por um objeto `GerenciadorCliente`, que corre numa thread própria. Isto permite que vários jogadores comuniquem com o servidor ao mesmo tempo, sem bloquear a execução principal.

7.2 Thread de Gestão da Sessão

A lógica global da partida é executada numa thread separada, associada ao `GerenciadorDeJogo`. Deste modo, o servidor consegue manter a aceitação inicial de clientes e, posteriormente, coordenar a sessão de forma independente.

7.3 Sincronização do Estado Partilhado

Como várias threads podem necessitar de aceder ao estado do jogo, foram utilizados métodos sincronizados na classe `EstadoJogo`. Esta abordagem evita inconsistências provocadas por acessos concorrentes ao mesmo estado partilhado.

A sincronização foi particularmente importante nas operações de:

- processamento de jogadas;
- atualização de tentativas;
- registo de vencedores;
- verificação do fim do jogo.

8. Cliente e Interação com o Utilizador

O cliente foi desenvolvido em modo textual, funcionando no terminal. A sua implementação encontra-se organizada de forma a permitir a receção assíncrona de mensagens do servidor e a introdução de jogadas por parte do utilizador.

A aplicação cliente utiliza duas componentes principais de execução:

- uma thread recetora, que lê continuamente as mensagens enviadas pelo servidor;
- a thread principal, que trata da leitura da jogada do utilizador e do respetivo envio.

No terminal, o cliente apresenta ao utilizador:

- a identificação do jogador;
- o estado da palavra;
- o número de tentativas restantes;
- as letras usadas;
- uma representação textual da forca;
- mensagens finais de vitória, derrota ou lotação máxima.

Esta solução permitiu criar uma interface simples, suficiente para demonstrar o funcionamento do jogo sem necessidade de recorrer a interfaces gráficas.

9. Robustez e Tratamento de Situações Especiais

Durante o desenvolvimento, foram considerados vários cenários de exceção.

9.1 Desconexão de Clientes

Caso um cliente se desligue inesperadamente, o servidor deteta essa situação e atualiza a lista de jogadores ativos. Se deixarem de existir jogadores ligados, a sessão pode ser terminada.

9.2 Rejeição de Novas Ligações

Após o início do jogo, o servidor deixa de aceitar novos jogadores. Se um novo cliente tentar entrar, é enviada uma mensagem informativa e a ligação é encerrada.

9.3 Ficheiro de Palavras Inexistente

Se o ficheiro externo de palavras não estiver disponível, o sistema continua funcional através de uma lista interna de palavras predefinidas.

Estas decisões contribuíram para aumentar a robustez global da aplicação.

10. Testes Realizados

Para validar o funcionamento do sistema, foram realizados vários testes funcionais, entre os quais:

- ligação simultânea de 2, 3 e 4 clientes;
- tentativa de ligação de um quinto cliente;
- arranque correto do jogo após o número mínimo de jogadores;
- atualização da máscara da palavra após jogadas corretas;

- decremento das tentativas após jogadas incorretas;
- deteção de vitória;
- deteção de derrota;
- envio das mensagens finais aos clientes;
- encerramento correto das ligações no final da sessão.

Os testes realizados permitiram confirmar o comportamento esperado da aplicação nas principais situações previstas.

11. Compilação e Execução

A compilação do projeto pode ser realizada através dos seguintes comandos:

Listing 1: Compilação do projeto

```
javac -d out src/module-info.java src/comum/*.java src/server/*.java src/client/*.
    java
```

Execução do servidor:

Listing 2: Execução do servidor

```
java --module-path out -m JogoDaForca/server.ServidorJogo
```

Execução do cliente:

Listing 3: Execução do cliente

```
java --module-path out -m JogoDaForca/client.ClienteJogo
```

Caso o cliente seja executado noutra máquina, deve ser utilizado o endereço IP da máquina onde o servidor está em execução.

12. Ligações do Projeto

- Repositório GitHub: Aceder ao repositório
- Vídeo de demonstração no YouTube: Ver demonstração no YouTube

13. Conclusão

O desenvolvimento deste projeto permitiu aplicar, em contexto prático, conceitos fundamentais da área de Sistemas Distribuídos, como comunicação em rede, concorrência, sincronização e gestão de estado partilhado.

A solução implementada demonstrou ser funcional, modular e adequada aos objetivos propostos. A utilização de uma arquitetura cliente-servidor permitiu separar claramente as responsabilidades entre

servidor e clientes, enquanto a divisão por classes tornou o código mais organizado e mais fácil de compreender.

Apesar de existirem possíveis extensões e melhorias futuras, considera-se que a aplicação desenvolvida constitui uma base sólida e coerente para um jogo da forca multijogador síncrono em Java.

14. Melhorias Futuras

Como evolução futura do trabalho, poderiam ser consideradas as seguintes melhorias:

- expansão do banco de palavras;
- adoção de uma interface gráfica;
- suporte para várias salas de jogo em simultâneo;
- reforço da aproximação integral ao protocolo sugerido no enunciado;
- criação de estatísticas de jogadores e partidas;
- melhoria da apresentação visual do cliente.